

07_vectors_multimodal

August 18, 2026

1 NB7 — Multimodal & Vectors: when one row weighs 10 MB

Stack: deltalake + DuckDB + NumPy. No model download, no vector DB, no API key. Maps to slide §11 (AI 2026: Multimodal, Vector & Agent) + deliverable bullet 7.

Parquet and Iceberg were designed for **KB-sized rows, read sequentially, in batches**. Multimodal AI breaks all three assumptions at once:

1. Rows become **MB** (images, clips, audio)
2. Access becomes **random** — thousands of clips/vectors per second
3. The GPU must be fed continuously; a starved GPU is money on fire

This notebook measures what actually breaks, then builds semantic search directly on a lakehouse table — and ends with the bug that matters most.

```
[1]: import _setup # noqa: F401

import time
from pathlib import Path

import duckdb
import numpy as np
import pyarrow as pa
import pyarrow.parquet as pq
from deltalake import DeltaTable, write_deltalake

from lakehouse import ROOT, du, human, path, reset

import generate_ai_data as gen

DOCS = path("bronze", "docs_multimodal")
if not Path(DOCS).exists():
    gen.main()

docs = DeltaTable(DOCS).to_pyarrow_table()
print(f"Corpus: {docs.num_rows:,} docs, embedding dim={len(docs.
    ↳column('emb')[0])}")
print(f"Columns: {docs.column_names}")
```

Corpus: 2,000 docs, embedding dim=256

Columns: ['doc_id', 'title', 'topic', 'subject_id', 'source', 'license', 'consent_train', 'generator', 'blob_uri', 'emb']

1.1 1. Inline blob vs pointer — measure, don't assume

The usual advice is “never put blobs in your table.” Let's test it properly instead of repeating it. Two layouts, same 200 media frames:

- **inline** — a blob column of raw bytes
- **pointer** — a blob_uri string; bytes live as separate objects

```
[2]: BLOB_DIR = Path(ROOT) / "blobs"
blob_files = sorted(BLOB_DIR.glob("*.bin"))
blobs = [f.read_bytes() for f in blob_files]
N = len(blobs)

inline_tbl = pa.table({
    "doc_id": pa.array(range(N), pa.int64()),
    "topic": [gen.TOPICS[i % len(gen.TOPICS)] for i in range(N)],
    "blob": pa.array(blobs, pa.binary()),
})
pointer_tbl = pa.table({
    "doc_id": pa.array(range(N), pa.int64()),
    "topic": [gen.TOPICS[i % len(gen.TOPICS)] for i in range(N)],
    "blob_uri": [f"blobs/{f.name}" for f in blob_files],
})

INLINE, POINTER = path("scratch", "media_inline"), path("scratch",
↳"media_pointer")
reset(INLINE, POINTER)
write_deltalake(INLINE, inline_tbl, mode="overwrite")
write_deltalake(POINTER, pointer_tbl, mode="overwrite")

print(f"inline table : {human(du(INLINE)):>10}")
print(f"pointer table: {human(du(POINTER)):>10} (+ {human(du(BLOB_DIR))} of
↳objects alongside)")
print(f"\nTotal bytes stored is nearly identical - the bytes have to live
↳somewhere.")
print("The difference is not SIZE. It is what a reader is forced to touch.")
```

```
inline table :    12.5 MB
pointer table:    4.7 KB  (+ 12.5 MB of objects alongside)
```

Total bytes stored is nearly identical - the bytes have to live somewhere.
The difference is not SIZE. It is what a reader is forced to touch.

1.1.1 Finding 1: column pruning already protects analytical scans

A query like `SELECT topic, count(*) GROUP BY topic` never mentions blob. Parquet is columnar, so it reads only the column chunks it needs.

We can prove it exactly from Parquet footer metadata — no timing required.

```
[3]: def column_bytes(table_path: str) -> dict[str, int]:
    """Compressed bytes per column, straight from the Parquet footer."""
    data_file = next(f for f in Path(table_path).rglob("*.parquet") if
    ↪ "_delta_log" not in f.parts)
    md = pq.ParquetFile(data_file).metadata
    out: dict[str, int] = {}
    for rg in range(md.num_row_groups):
        for c in range(md.row_group(rg).num_columns):
            col = md.row_group(rg).column(c)
            out[col.path_in_schema] = out.get(col.path_in_schema, 0) + col.
    ↪ total_compressed_size
    return out

inline_cols = column_bytes(INLINE)
pointer_cols = column_bytes(POINTER)
projected = ["doc_id", "topic"]

inline_scan = sum(inline_cols[c] for c in projected)
pointer_scan = sum(pointer_cols[c] for c in projected)
print("Bytes read by `SELECT topic, count(*) GROUP BY topic`:")
print(f"  inline layout : {human(inline_scan)} of {human(sum(inline_cols.
    ↪ values()))} total")
print(f"  pointer layout: {human(pointer_scan)} of {human(sum(pointer_cols.
    ↪ values()))} total")
print(f"\n→ The blob column costs the analytical scan essentially NOTHING.")
print("  Projection pushdown is real. The scary advice is wrong for this case.")
```

Bytes read by `SELECT topic, count(\*) GROUP BY topic`:

```
  inline layout : 1.2 KB  of 12.5 MB total
  pointer layout: 1.2 KB  of 2.4 KB total
```

→ The blob column costs the analytical scan essentially NOTHING.
Projection pushdown is real. The scary advice is wrong for this case.

1.1.2 Finding 2: random single-row access is where it actually breaks

Now fetch **one** frame: `WHERE doc_id = 137`.

Parquet's unit of I/O is the **row group**, not the row. To get one blob you must read (and decompress) the whole row group that contains it. With a pointer you issue one **GET** for exactly those bytes.

This — not file size — is the number Lance’s “3–35× faster random access” claim is about.

```
[4]: def row_group_bytes(table_path: str) -> tuple[int, int, int]:
    data_file = next(f for f in Path(table_path).rglob("*.parquet") if
        ↪ "_delta_log" not in f.parts)
    md = pq.ParquetFile(data_file).metadata
    rg0 = md.row_group(0)
    return md.num_row_groups, rg0.num_rows, rg0.total_byte_size

n_rg, rg_rows, rg_bytes = row_group_bytes(INLINE)
one_blob = len(blobs[0])

print(f"inline parquet: {n_rg} row group(s), {rg_rows} rows, {human(rg_bytes)}_
    ↪ per group")
print(f"\nFetching ONE frame (doc_id=137):")
print(f"  inline → must read the row group: {human(rg_bytes)}")
print(f"  pointer → one GET of the object: {human(one_blob)}")
print(f"  → amplification: {rg_bytes / one_blob:.0f}× more bytes than needed")
print(f"\nAt 1,000 random frame fetches/sec to feed a GPU, that amplification_
    ↪ IS")
print(f"the GPU-starvation problem. Formats like Lance restructure the file so_
    ↪ a")
print(f"random read costs ~one row, not one row group.")

AMPLIFICATION = rg_bytes / one_blob
```

inline parquet: 1 row group(s), 200 rows, 12.5 MB per group

Fetching ONE frame (doc_id=137):
 inline → must read the row group: 12.5 MB
 pointer → one GET of the object: 64.0 KB
 → amplification: 200× more bytes than needed

At 1,000 random frame fetches/sec to feed a GPU, that amplification IS
the GPU-starvation problem. Formats like Lance restructure the file so a
random read costs ~one row, not one row group.

1.2 2. Embeddings as a column: the storage arithmetic

The slide’s numbers: VECTOR(768, FLOAT) = 3,072 B/row; INT8 = 768 B/row. Let’s verify the 4× on real files rather than trusting the arithmetic.

```
[5]: emb = np.array(docs.column("emb").to_pylist(), dtype="float32")
n, dim = emb.shape
print(f"embeddings: {n:,} × {dim} ({human(emb.nbytes)} in memory)")
print(f"per row: float32 = {dim * 4:,} B  int8 = {dim * 1:,} B (4× smaller)")
```

```

print(f"at dim=768: float32 = {768 * 4:,} B    int8 = {768:,} B    ← the slide's
↪numbers")

# int8 symmetric quantization: embeddings are unit vectors, so values  [-1, 1]
SCALE = 127.0
emb_i8 = np.clip(np.round(emb * SCALE), -127, 127).astype("int8")

f32_tbl = pa.table({"doc_id": docs.column("doc_id"), "emb": docs.column("emb")})
i8_tbl = pa.table({
    "doc_id": docs.column("doc_id"),
    "emb": pa.FixedSizeListArray.from_arrays(pa.array(emb_i8.ravel(), pa.
↪int8()), dim),
})
F32, I8 = path("scratch", "emb_f32"), path("scratch", "emb_int8")
reset(F32, I8)
write_deltalake(F32, f32_tbl, mode="overwrite")
write_deltalake(I8, i8_tbl, mode="overwrite")

print(f"\nOn disk (Parquet, compressed):")
print(f"  float32: {human(du(F32))}")
print(f"  int8    : {human(du(I8))}    → {du(F32) / max(du(I8), 1):.1f}×
↪smaller")

```

embeddings: 2,000 × 256 (2.0 MB in memory)
 per row: float32 = 1,024 B int8 = 256 B (4× smaller)
 at dim=768: float32 = 3,072 B int8 = 768 B ← the slide's numbers

On disk (Parquet, compressed):
 float32: 2.6 MB
 int8 : 451.9 KB → 5.8× smaller

1.3 3. Semantic search *is* a SQL query now

DuckDB ships `array_cosine_similarity` in core — no extension, no download, no network. The embedding lives in the table, so retrieval is a join away from every other column you own (topic, license, consent, timestamps).

```

[6]: con = duckdb.connect()
con.register("docs", docs)

# Gotcha worth knowing: we WROTE `fixed_size_list<float>[256]`, but the Delta
# protocol has no fixed-width vector type - only `array<element>`. So the column
# comes back as a variable-length `list<float>` and we must cast it before
# DuckDB's fixed-size array functions will bind:
print("arrow type on read:", docs.schema.field("emb").type, " → cast to
↪FLOAT[dim] at query time")
print("This missing type is exactly why Hudi 1.2 added a first-class")

```

```

print("VECTOR(dim, type) column, and why the slide flags it as the 2026 trend.
↪\n")

query_vec = emb[7].tolist()           # pretend this came from an encoder
t0 = time.perf_counter()
hits = con.sql(f"""
    SELECT doc_id, title, topic,
           array_cosine_similarity(emb::FLOAT[{dim}], {query_vec}::
↪FLOAT[{dim}]) AS sim
    FROM docs
    ORDER BY sim DESC
    LIMIT 5
""").fetchall()
sql_ms = (time.perf_counter() - t0) * 1000

print(f"Query doc: {docs.column('title')[7]} (topic={docs.
↪column('topic')[7]})")
print(f"\n{'doc_id':>7} {'topic':<12} {'sim':>6} title")
for doc_id, title, topic, sim in hits:
    print(f"{'doc_id':>7} {'topic':<12} {'sim':6.3f} {'title}'")
print(f"\nbrute-force scan over {n:,} vectors: {sql_ms:.1f} ms")

```

arrow type on read: list<element: float> → cast to FLOAT[dim] at query time
This missing type is exactly why Hudi 1.2 added a first-class
VECTOR(dim, type) column, and why the slide flags it as the 2026 trend.

Query doc: storage-note-00007 (topic=storage)

doc_id	topic	sim	title
7	storage	1.000	storage-note-00007
1703	storage	0.779	storage-note-01703
1200	storage	0.777	storage-note-01200
766	storage	0.776	storage-note-00766
1250	storage	0.768	storage-note-01250

brute-force scan over 2,000 vectors: 13.0 ms

1.3.1 Filtered semantic search — the thing a standalone vector DB struggles with

“Find similar docs **that we are actually licensed to train on**” is one query here, because the vector and the governance columns are in one row.

```

[7]: legal = con.sql(f"""
    SELECT doc_id, topic, license,
           array_cosine_similarity(emb::FLOAT[{dim}], {query_vec}::
↪FLOAT[{dim}]) AS sim
    FROM docs
    WHERE consent_train AND license <> 'unknown'

```

```

ORDER BY sim DESC
LIMIT 5
""").fetchall()
for doc_id, topic, lic, sim in legal:
    print(f"{doc_id:>7} {topic:<12} {lic:<12} {sim:6.3f}")
print("\nNo sync job, no ID reconciliation, no 'is this vector still valid?'")

```

1703	storage	proprietary	0.779
1200	storage	user-owned	0.777
766	storage	commercial	0.776
1250	storage	user-owned	0.768
570	storage	commercial	0.768

No sync job, no ID reconciliation, no 'is this vector still valid?'

1.3.2 Honest scaling check: brute force is not a serving path

The slide is explicit — in-table brute force suits *analytical* semantic queries and *offline* recall measurement, not sub-100 ms online serving.

```

[8]: for size in (n, n * 50, n * 500):
    est = sql_ms * size / n
    verdict = "fine" if est < 100 else ("borderline" if est < 1000 else "NOT a_
    ↪serving path")
    print(f" {size:>10,} vectors → ~{est:8.1f} ms {verdict}")
print("\nRule from the slide: vector DB = a rebuildable DERIVED INDEX.")
print("
    lakehouse = the SYSTEM-OF-RECORD.")

```

2,000	vectors → ~	13.0 ms	fine
100,000	vectors → ~	649.5 ms	borderline
1,000,000	vectors → ~	6494.6 ms	NOT a serving path

Rule from the slide: vector DB = a rebuildable DERIVED INDEX.
lakehouse = the SYSTEM-OF-RECORD.

1.4 4. What does int8 quantization cost you in recall?

4× less storage is only a win if retrieval quality survives. Measure it: float32 top-10 is ground truth, int8 top-10 is the candidate.

```

[9]: def topk(matrix: np.ndarray, queries: np.ndarray, k: int = 10) -> np.ndarray:
    sims = queries @ matrix.T
    return np.argsort(-sims, axis=1)[: , :k]

rng = np.random.default_rng(0)
q_idx = rng.choice(n, size=100, replace=False)
emb_i8_deq = emb_i8.astype("float32") / SCALE
emb_i8_deq /= np.linalg.norm(emb_i8_deq, axis=1, keepdims=True)

```

```

gold = topk(emb, emb[q_idx])
cand = topk(emb_i8_deq, emb_i8_deq[q_idx])
recall = np.mean([len(set(g) & set(c)) / len(g) for g, c in zip(gold, cand)])

# Exact-ID recall is harsh: it counts a swap between two docs that are equally
# relevant as a miss. For RAG, what usually matters is whether the retrieved
# docs are still ABOUT the right thing. Measure both.
topics_arr = np.array(docs.column("topic").to_pylist())
topic_fidelity = np.mean([
    (topics_arr[c] == topics_arr[q]).mean() for q, c in zip(q_idx, cand)
])

print(f"recall@10 (exact doc IDs), int8 vs float32: {recall:.3f}")
print(f"topic fidelity of int8 top-10: {topic_fidelity:.3f}")
print(f"storage saved: {(1 - du(I8) / du(F32)) * 100:.0f}%")
print(f"""
→ int8 loses ~{(1 - recall) * 100:.0f}% of exact IDs but {topic_fidelity * 100:.0f}% of results are still
on-topic. The "misses" are swaps between near-equivalent neighbours, which
is why exact-ID recall UNDERSTATES quantization quality for RAG.
Measure both on YOUR corpus before shipping - the trade is corpus-dependent.
↪""")

```

```

recall@10 (exact doc IDs), int8 vs float32: 0.904
topic fidelity of int8 top-10: 1.000
storage saved: 83%

```

```

→ int8 loses ~10% of exact IDs but 100% of results are still
on-topic. The "misses" are swaps between near-equivalent neighbours, which
is why exact-ID recall UNDERSTATES quantization quality for RAG.
Measure both on YOUR corpus before shipping - the trade is corpus-dependent.

```

1.5 5. The lifecycle bug — the real reason embeddings belong in the table

*“When a row is deleted, expires, is edited or reprocessed, the embedding must follow exactly that lifecycle. Every warehouse → vector-DB sync pipeline is a lifecycle-skew bug waiting to happen — and the moment you get a right-to-forget request, it becomes a **compliance** bug.”*

Let’s produce that bug on purpose.

First, a nightly sync builds an external vector index (a copy):

```

[10]: EXTERNAL = path("scratch", "vector_index_external")
reset(EXTERNAL)
write_deltalake(EXTERNAL, pa.table({
    "doc_id": docs.column("doc_id"),

```

```

    "emb": docs.column("emb"),
  }), mode="overwrite")
print(f"External index synced: {DeltaTable(EXTERNAL).count():,} vectors")

```

External index synced: 2,000 vectors

1.5.1 A data subject exercises their right to erasure

user_042 asks for deletion. We delete from the lakehouse — the system-of-record — as we should.

```

[11]: SUBJECT = "user_042"
      INTABLE = path("scratch", "docs_intable")
      reset(INTABLE)
      write_deltalake(INTABLE, docs, mode="overwrite")

      dt = DeltaTable(INTABLE)
      victim_ids = con.sql(f"SELECT doc_id FROM docs WHERE subject_id = '{SUBJECT}'").
        ↪fetchall()
      victim_ids = [r[0] for r in victim_ids]
      dt.delete(f"subject_id = '{SUBJECT}'")

      print(f"Erasure request for {SUBJECT}: {len(victim_ids)} docs")
      print(f"lakehouse rows: {docs.num_rows:,} → {DeltaTable(INTABLE).count():,}")
      print(f"external index rows: {DeltaTable(EXTERNAL).count():,} ← untouched")

```

Erasure request for user_042: 8 docs

lakehouse rows: 2,000 → 1,992

external index rows: 2,000 ← untouched

1.5.2 Now query both

```

[12]: victim_vec = emb[victim_ids[0]].tolist()

      con.register("intable", DeltaTable(INTABLE).to_pyarrow_table())
      con.register("external", DeltaTable(EXTERNAL).to_pyarrow_table())

      in_hits = con.sql(f"""SELECT count(*) FROM intable
                          WHERE doc_id IN ({','.join(map(str, victim_ids))})""").
        ↪fetchone()[0]
      ex_hits = con.sql(f"""SELECT count(*) FROM external
                          WHERE doc_id IN ({','.join(map(str, victim_ids))})""").
        ↪fetchone()[0]

      print(f"Erased docs still retrievable from the lakehouse: {in_hits}")
      print(f"Erased docs still retrievable from the external index: {ex_hits} ←␣
        ↪VIOLATION")
      print(f"""
      The external index will happily return {SUBJECT}'s content to a RAG prompt

```

```
until the next sync - and if the sync is one-way upsert (the common case),
*forever*, because deletes are the operation sync pipelines forget.
"""
```

```
Erased docs still retrievable from the lakehouse:      0
Erased docs still retrievable from the external index: 8  ← VIOLATION
```

The external index will happily return user_042's content to a RAG prompt until the next sync - and if the sync is one-way upsert (the common case), **forever**, because deletes are the operation sync pipelines forget.

1.5.3 The correct propagation mechanism: Change Data Feed

If you *must* keep a derived index, do not re-sync the whole table. Read the **change feed** so deletes propagate as first-class events.

```
[13]: CDF_TABLE = path("scratch", "docs_cdf")
reset(CDF_TABLE)
write_deltalake(CDF_TABLE, pa.table({"doc_id": docs.column("doc_id"),
                                     "subject_id": docs.column("subject_id")}),
               mode="overwrite", configuration={"delta.enableChangeDataFeed":
               ↪ "true"})
DeltaTable(CDF_TABLE).delete(f"subject_id = '{SUBJECT}'")

cdf = DeltaTable(CDF_TABLE).load_cdf(starting_version=1).read_all()
changes = cdf.column("_change_type").to_pylist()
deletes = [t for t in changes if t == "delete"]
print(f"CDF rows since v1: {len(changes)}   deletes: {len(deletes)}")
print(f>Delete events carry the doc_ids to evict: {cdf.column('doc_id').
  ↪to_pylist()[:5]} ...)
print("""
That is the contract: the index subscribes to deletes instead of guessing.
Best of all is not needing the sync - keep the vector in the row (§2 above)
and the lifecycle is enforced by the table itself.
""")
```

```
CDF rows since v1: 8   deletes: 8
Delete events carry the doc_ids to evict: [42, 292, 542, 792, 1042] ...
```

That is the contract: the index subscribes to deletes instead of guessing. Best of all is not needing the sync - keep the vector in the row (§2 above) and the lifecycle is enforced by the table itself.

1.6 NB7 pass criteria

Check	Target
Random-access amplification measured	5× more bytes for inline blob
int8 quantization	3× smaller on disk
int8 recall@10	0.80 vs float32 (typically ~0.90)
int8 topic fidelity	0.95 — the metric that matters for RAG
Semantic search returns same-topic neighbours	top-5 majority share the query's topic
Lifecycle bug reproduced	0 hits in-table, > 0 hits in the external index
CDF emits delete events	1 delete in the change feed

```
[14]: top_topics = [h[2] for h in hits]
query_topic = docs.column("topic")[7].as_py()

checks = {
    "random-access amplification 5x": AMPLIFICATION >= 5,
    "int8 3x smaller": du(F32) / max(du(I8), 1) >= 3,
    "int8 recall@10 0.80": recall >= 0.80,
    "int8 topic fidelity 0.95": topic_fidelity >= 0.95,
    "top-5 share query topic": top_topics.count(query_topic) >= 3,
    "lifecycle bug reproduced": in_hits == 0 and ex_hits > 0,
    "CDF emits delete events": len(deletes) == len(victim_ids),
}
for k, v in checks.items():
    print(f" [{'PASS' if v else 'FAIL'}] {k}")
assert all(checks.values()), "NB7 incomplete - see FAIL rows above"
print("\nNB7 complete.")
```

```
[PASS] random-access amplification 5x
[PASS] int8 3x smaller
[PASS] int8 recall@10 0.80
[PASS] int8 topic fidelity 0.95
[PASS] top-5 share query topic
[PASS] lifecycle bug reproduced
[PASS] CDF emits delete events
```

NB7 complete.